

Prime Automation Framework v2 – Developer & Tester Guide

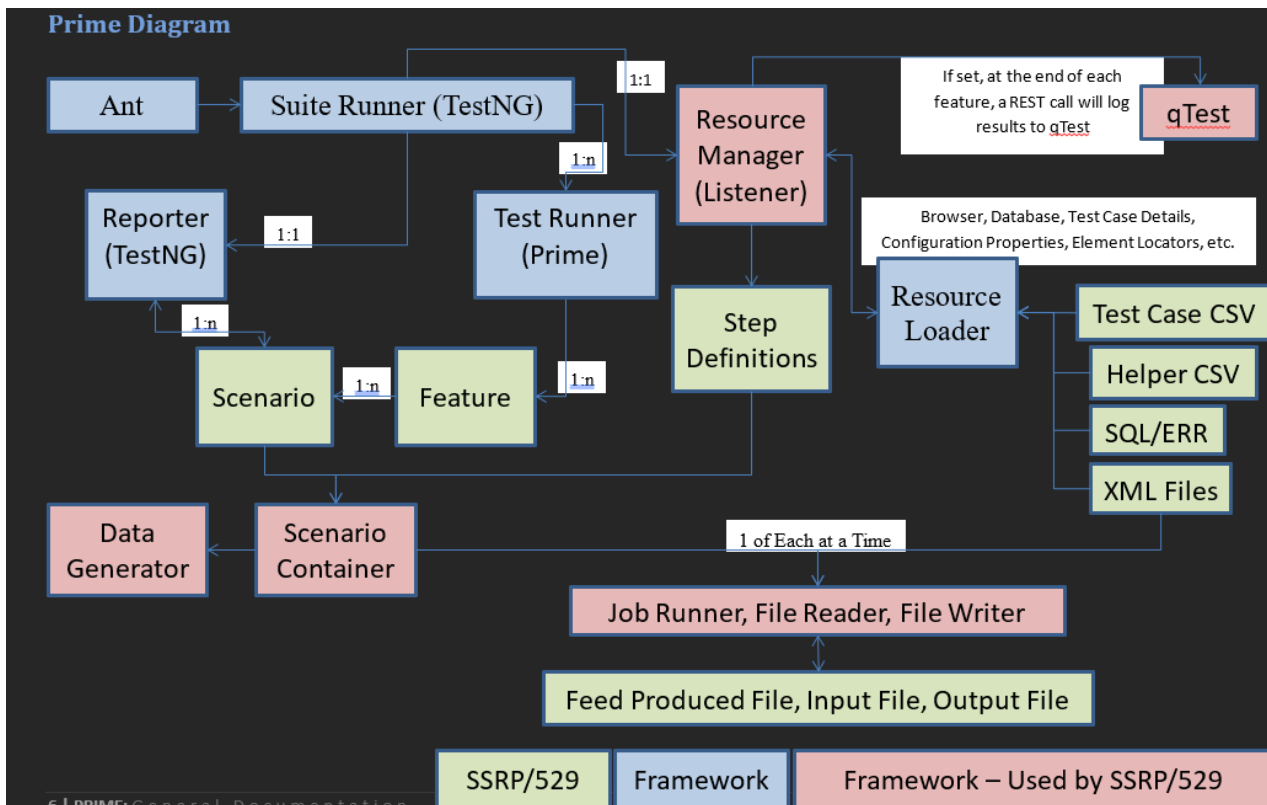
Purpose

Prime v2 is a **Cucumber + TestNG based Java automation framework** designed for the 529 / ABLE (Unite) systems. Astro also uses same framework as a base.

It unifies UI (front-office) and DB (back-office) testing with CSV-driven data, SQL validation, and HTML reports.

Tech Stack: Cucumber • Selenium • TestNG • Java • CSV • SQL • Reports

1. Framework Architecture Overview



What Is Prime v2?

- Prime v2 is a comprehensive, data-driven test automation framework designed to streamline regression testing for 529 and ABLE systems, Astro
- It's built on industry-standard tools like Cucumber, Selenium, TestNG, and Java, providing a unified platform for both UI and database validation
- The framework eliminates the complexity of managing multiple testing approaches by offering a single, cohesive solution that scales with your testing needs
- Key features include:
 - End-to-end automation with complete test coverage from UI to backend
 - Seamless integration of frontend and backend testing through UI and DB validation
 - Flexible data management for complex scenarios using CSV and SQL data-driven approaches
 - Comprehensive reporting with HTML output and integration with qTest



End-to-End Automation

Complete test coverage from UI to backend



UI + DB Validation

Seamless integration of frontend and backend testing



CSV + SQL Data-Driven

Flexible data management for complex scenarios



HTML + qTest Reporting

Comprehensive reporting with qTest integration

Simple Flow

ANT TestNG Suite Runner PrimeResourceManager ResourceLoader

Feature File (.feature) Step Definitions (.java)

CSV / SQL / XML Data ScenarioContainer

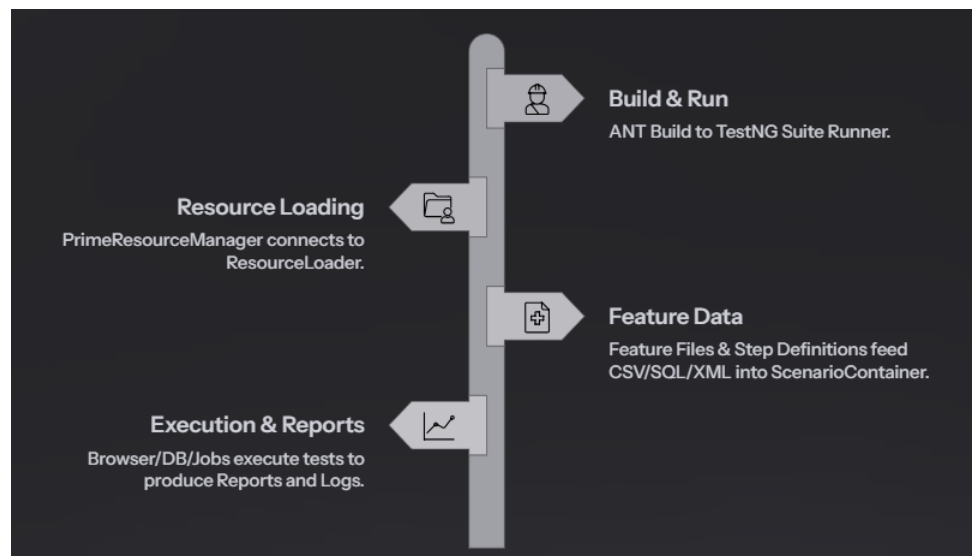
Browser / DB / Jobs Test Execution Reports + Logs

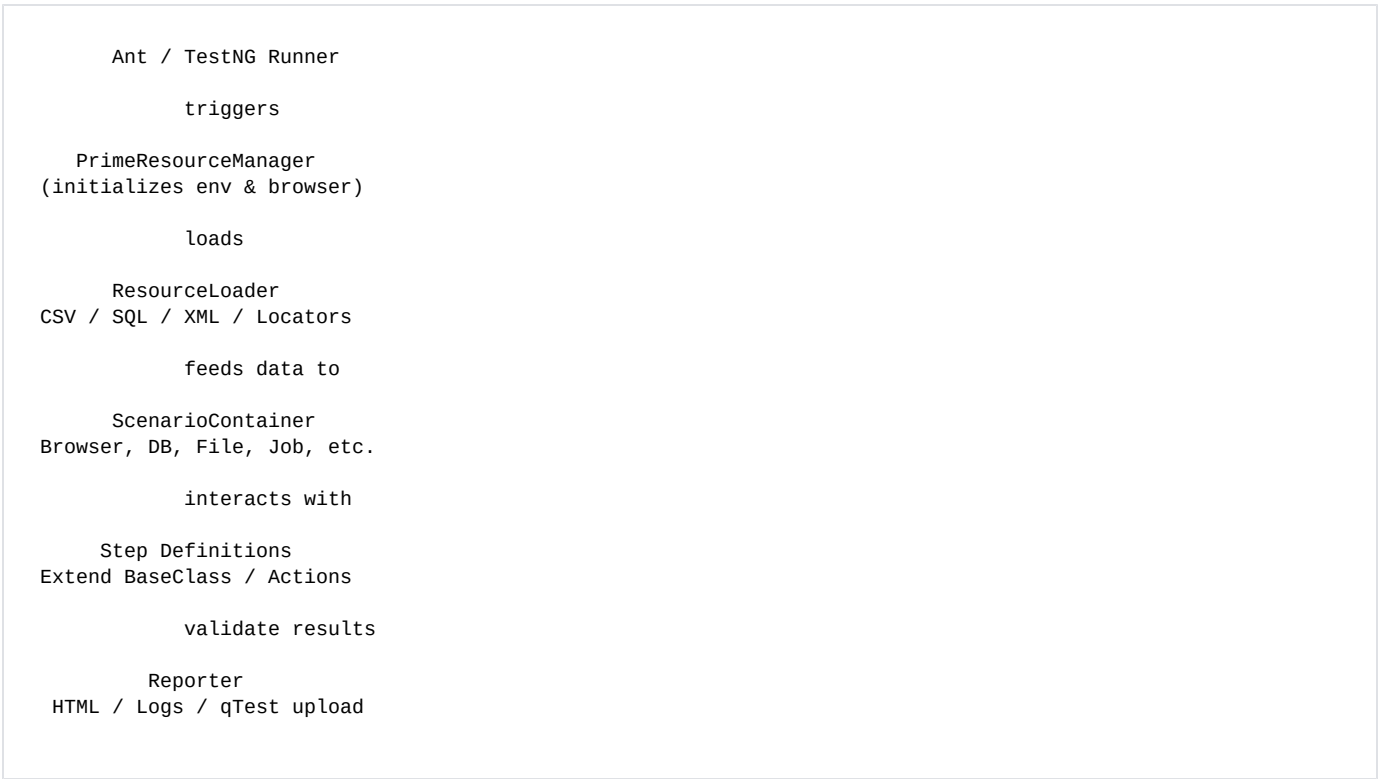
Framework Architecture Overview:

- The Prime v2 framework follows a **modular architecture that separates concerns and promotes reusability**
- This design enables easy maintenance, scalability, and allows multiple team members to work efficiently on different test scenarios simultaneously
- The key architectural components are:
 - Build & Run: ANT orchestrates the build process and triggers TestNG suite execution
 - Resource Management: PrimeResourceManager initializes resources and loads configuration data
 - Test Execution: Feature files paired with step definitions execute test scenarios
 - Reporting: Results captured and formatted into comprehensive HTML reports
- This modular approach allows the framework to be easily extended and customized to meet evolving testing needs

Framework Flow Diagram

- The framework flow diagram provides a high-level overview of how the test automation framework operates, from build and execution to reporting.
- It shows the key components involved and how data flows through the system, helping developers understand the end-to-end process.
- The build and run stage starts with the ANT build tool, which feeds into the TestNG suite runner to manage test execution.
- The resource loading stage involves the PrimeResourceManager connecting to the ResourceLoader to ensure proper resource allocation.
- The feature data stage shows how the feature files and step definitions feed data from CSV, SQL, and XML sources into the ScenarioContainer.
- Finally, the execution and reports stage depicts how the browser, database, and other jobs execute the tests, producing the final reports and logs.
- Understanding this flow is critical for troubleshooting issues and optimizing the performance of the test automation framework.





2. Folder Structure Overview:

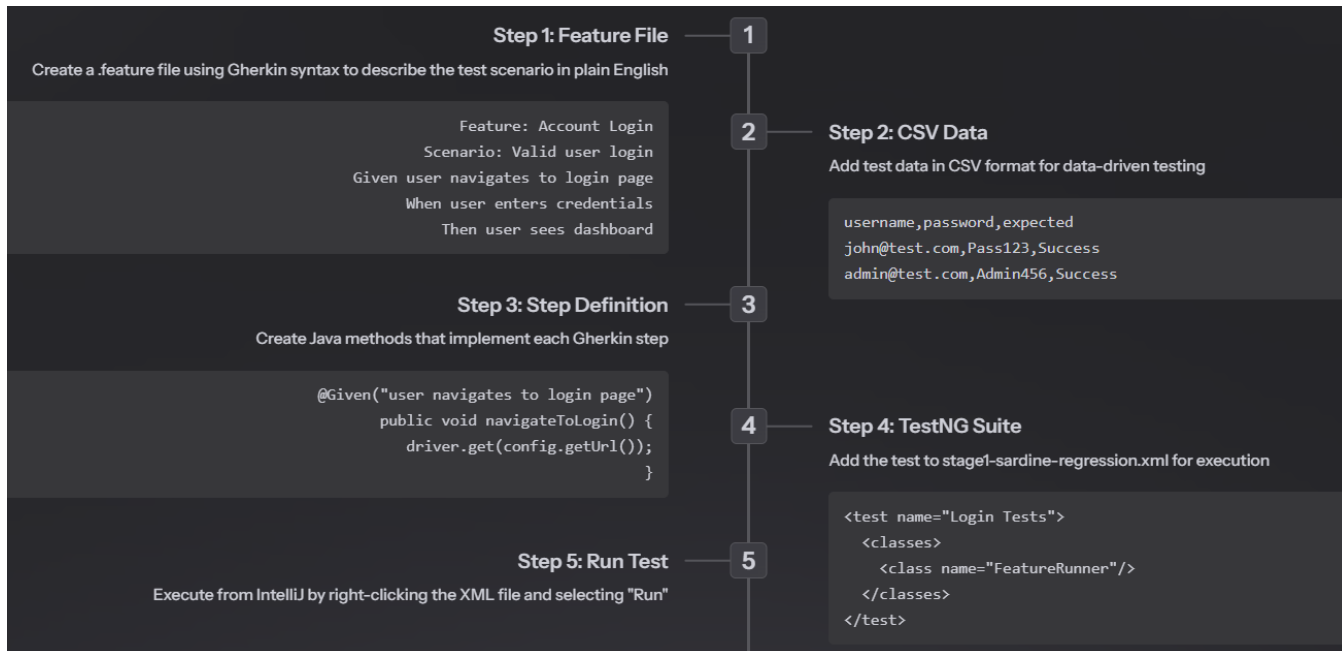
Folder	Purpose
prime/source/com/cs529/qa/prime/core/BaseClass.java	Core Engine – Contains the framework’s core components including PrimeResourceManager, ResourceLoader, DB connections and base utilities
/<Project>/testsuite/frontoffice/	UI Feature Files – Cucumber feature files defining user-facing test scenarios and acceptance criteria, SQL queries, XML configurations, and backend validation files
/<Project>/config	Environment Configs – Environment-specific properties, database connections, and application URLs
/source/com/cs529/qa/unite/frontoffice/	Step Definitions – Java implementations mapping Gherkin steps to executable code
/reports/	Test Reports – Generated HTML reports, logs, and screenshots from test execution, also can be located at C:/Reporter/

3. Key Framework Components

Component	Role
PrimeResourceManager	The PrimeResourceManager is the heart of the framework, responsible for initializing browser instances, establishing database connections, and performing cleanup operations to prevent resource leaks.
ResourceLoader	The ResourceLoader dynamically loads and parses data from multiple sources like CSV files, SQL databases, and XML configurations, providing a unified interface for accessing test data.
BaseClass	The BaseClass and StepDefs provide reusable step definitions and common test utilities, with helper methods for navigation, validation, and data manipulation.
ScenarioContainer	Handles browser, DB, file reader/writer, data storage.
FeatureRunner / ParallelFeatureRunner	The FeatureRunner orchestrates the execution of Cucumber scenarios, managing test flow, ensuring proper execution order, and collecting results for reporting.
Reporter	The Reporter generates comprehensive HTML reports, integrates with qTest for centralized test management, captures screenshots on failures, and provides visual dashboards for test metrics and trends.

4. Creating a New Test Case (Step-by-Step)

- The slide outlines a 5-step process for creating a new test case in Prime v2
- Step 1 is to create a .feature file using Gherkin syntax to describe the test scenario in plain English
- Step 2 is to add test data in CSV format for data-driven testing
- Step 3 is to create Java methods that implement each Gherkin step
- Step 4 is to add the test to the stage1-sardine-regression.xml suite for execution
- Step 5 is to execute the test from IntelliJ by right-clicking the XML file and selecting "Run"
- This modular approach allows QA analysts and developers to work on different layers simultaneously, maximizing productivity



Step 1 – Create Feature File

```
/frontoffice/<plan>/enrollment/feature/Enrollment.feature
```

```
Feature: Enrollment Workflow – CA Direct
Scenario: Enroll a new participant
  Given starting enrollment URL for member
  When user enters valid prospect details
  Then verify enrollment completes successfully
```

Step 2 – Add CSV Test Data

```
/frontoffice/<plan>/enrollment/data/Enrollment.csv
```

```
TestCase,FirstName,LastName,SSN,ContributionType,ExpectedResult
ENR_TC01,John,Doe,123456789,Check,Enrollment Successful
```

Step 3 – Link Step Definitions

```
/stepdefs/enrollment/EnrollmentSteps.java
```

```
@Given("^starting enrollment URL for member$")
public void launchEnrollmentPage() {
    driver.get(config.getPlanURL("cadirect"));
}
```

Step 4 – Add to TestNG Suite

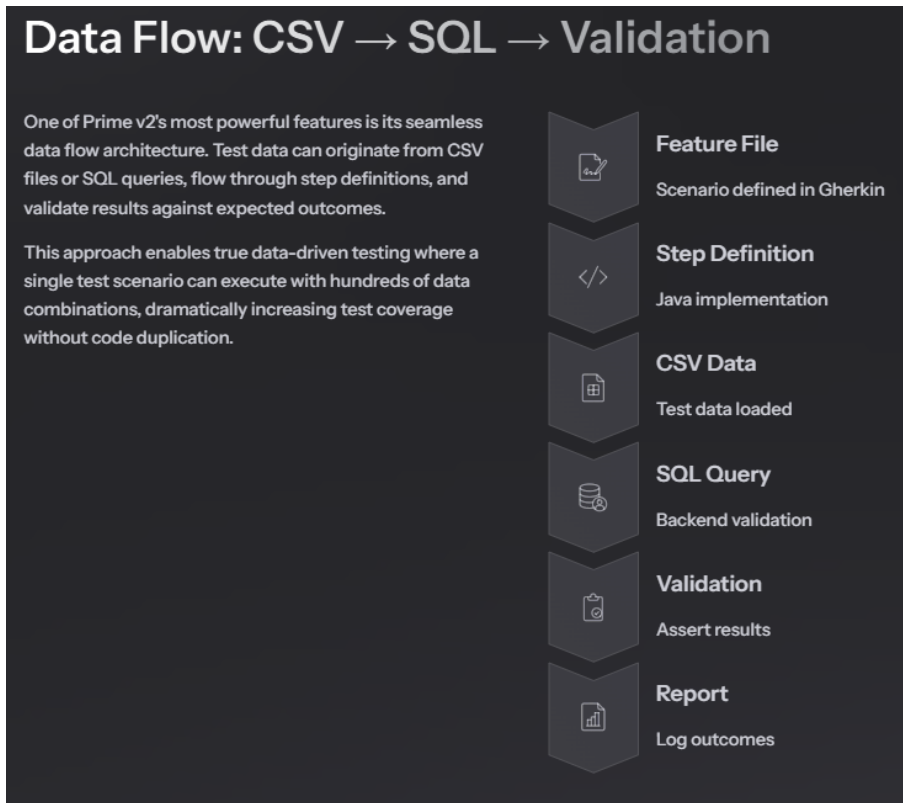
```
stage1-sardine-regression.xml

<test name="CA Direct - Enrollment Regression">
  <parameter name="traunch" value="cadirect"/>
  <parameter name="tags" value="@regression and @enrollment"/>
  <parameter name="features" value="frontoffice/enrollment/feature/Enrollment.feature"/>
  <classes>
    <class name="com.cs529.qa.prime.runner.FeatureRunner"/>
  </classes>
</test>
```

Step 5 – Run from IntelliJ

1. Open **Ant Tool Window** run build.xml
2. Or right-click the TestNG suite **Run with TestNG**
3. Reports saved under C:\Reporter\Prime\<timestamp>\<Computer_name>\index.html

5. CSV SQL Validation Flow



6. Running Regression & Reports

Task	Command / Action	Output
Clean Build	• cd ~\unite-test-automation\unite • ant cleanbuild	Recompiles Prime + Unite
Run Suite	• cd bin • ant	Executes test cases

Debug Mode	Set target debug in build.xml under \unite folder	Waits for IntelliJ remote debug (8888)
Report Path	C:\Reporter\Prime	HTML + Logs per date/time
Rerun Failed Tests	Use PrimeRerun.txt with RerunFailedTestcaseRunner	Runs only failed scenarios

7. Environment Setup Checklist

- Environment variables are critical for our test framework to function properly across different machines and CI/CD environments
- We need to ensure all required variables are correctly configured before running any tests

Variable	Example	Purpose
ANT_HOME	C:\ant\ant-1.9.4	Ant execution
CUCUMBER_HOME	C:\cucumber\4.2.0	BDD support
SELENIUM_HOME	C:\selenium\selenium-3.141.59	Grid path
TESTNG_HOME	C:\testng\testng-6.8	TestNG engine
PRIME_HOME	C:\work\unite-test-automation\unite	Project root
JAVA_HOME	C:\Program Files\Java\jdk1.8.0_40	Java 8 runtime V2 is upgraded to Java 17 so it can work with Java 17+
PATH	%JAVA_HOME%\bin;%ANT_HOME%\bin;%SELENIUM_HOME%	CLI access

Verify with set in cmd to confirm paths.

8. Navigation in IntelliJ - Useful Shortcuts

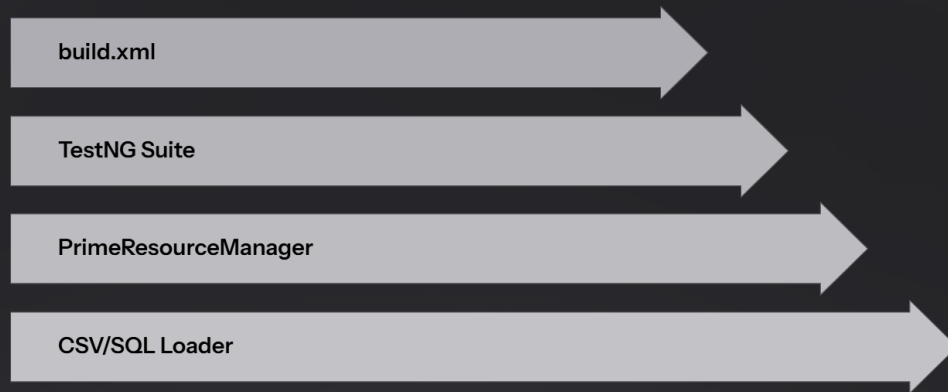
Action	Shortcut
Go to Definition from Feature	Ctrl + Click
Find in Project	Ctrl + Shift + F
Open File/Class	Ctrl + N / Ctrl + Shift + N
Run Suite	Right-click XML Run TestNG
Rebuild Project	Ctrl + F9
Ant Execution Panel	View Tool Windows Ant

9. Simple Execution Flowchart

- The build phase is the first step in the regression execution flow, where ANT reads the build.xml file and compiles the source code, resolves dependencies, and prepares the test environment.
- This phase ensures that all the necessary components are available before the test execution begins, setting the stage for the rest of the process.
- The execution phase follows the build phase, where TestNG orchestrates the test execution based on the suite configuration.
- The PrimeResourceManager initializes the browsers and databases, and the data loaders provide the necessary test data.
- Each scenario runs independently with its own context and resources, allowing for a structured and organized test execution process.
- Understanding this complete execution flow helps in troubleshooting issues and optimizing the test performance.

Regression Execution Flow

Understanding the complete execution flow helps troubleshoot issues and optimize test performance. Here's how a typical regression run progresses from build to reporting.



Build Phase

ANT reads build.xml and compiles source code, resolves dependencies, and prepares the test environment. This phase ensures all necessary components are available before test execution begins.

Execution Phase

TestNG orchestrates test execution based on suite configuration, PrimeResourceManager initializes browsers and databases, and data loaders provide test data. Each scenario runs independently with its own context and resources.

```
build.xml
  runs
testng suite
(Regression)
  invokes
PrimeResource
Manager
  loads
CSV / SQL /
Feature files
  executes
Browser + DB
  outputs
HTML Reports
```

10. Reporting & Jenkins Integration:

Comprehensive Reporting

Prime v2 generates detailed HTML reports after each test execution, providing visual dashboards with test results, execution time, pass/fail statistics, and failure screenshots. Reports are automatically timestamped and organized for easy tracking of trends over time.

CI/CD Integration

The framework seamlessly integrates with Jenkins for automated daily regression runs. Jenkins jobs can be configured to trigger tests on code commits, scheduled intervals, or manual requests. Failed tests can be automatically rerun to identify intermittent issues.

qTest Integration

Test results are pushed to qTest for centralized test management, enabling traceability between requirements, test cases, and defects across the entire QA lifecycle.

11. Common References

- Automation Standard Practices.docx Coding & naming rules
- Folder Structure and Naming.docx File hierarchy guidelines
- General-Documentation.docx Prime core logic & component flows
- Environment-Setup.docx Local variable setup
- Front-End Standards.docx Feature file syntax & UI locators
- Compile-Run.docx Ant execution & debugging
- Doc Link: <https://gitlab.com/ascensus-gs/products/depot/qa-automation/automation/-/tree/6b989f3125a65ab2f61d02a7105c19edf286989a/prime/docs>

12. Design Philosophy & Best Practices

Framework Design Principles

Prime v2 adheres to software engineering best practices including separation of concerns, DRY (Don't Repeat Yourself), and SOLID principles. This ensures the codebase remains maintainable as test suites grow.

Test Maintainability

By separating test logic (step definitions) from test data (CSV/SQL) and test scenarios (feature files), changes to any layer don't cascade to others. This isolation dramatically reduces maintenance costs.

Continuous Improvement

The framework evolves based on team feedback and emerging testing needs. Regular retrospectives identify pain points and opportunities for enhancement, ensuring the framework remains relevant and effective.

Community & Support

Leverage the internal QA community for support, share reusable step definitions, and contribute to the framework's component library. Collaboration amplifies everyone's productivity.